

LLD Notes — Phase 3, Part 2

Builder · Observer · Jumbled exercise 1 | Concepts, your doubts, your mistakes, interview answers

Colour key. Blue = a doubt you asked · Red = a mistake you made · Green = rule to remember · Amber = how to say it in an interview.

Contents

6. Builder — the two broken alternatives · structure · where each rule goes · immutability · defensive copy · thread safety · when not to · exercise 20
7. Observer — the problem · structure · push vs pull · the event object · isolation · async · CopyOnWriteArrayList · lapsed listener · when not to · Spring events · exercise 21
8. Jumbled exercise 1 (co-working booking) — the decisions, the access-card trap, the corrected build, the pressure answers
9. Updated mistake summary · requirement → pattern cheat sheet · interview quick-recall · pre-submit checklist

6. Builder

The problem: two alternatives, both broken

Some objects have **a few required fields and many optional ones**. Two obvious approaches both fail.

Attempt 1 — the big constructor

```
new RideRequest("U1", "Airport", "Office", "SEDAN", false, 1, null, true, null, "call on arrival");
//                                     ↑share?           ↑pet?
```

- **Same-typed parameters can be swapped silently.** `shareRide` and `petFriendly` are both boolean — swap them and it **still compiles**. The rider gets a shared ride with no pet allowed, and no tool will ever catch it. Same for `pickup/drop` (both Strings).
- **Unreadable at the call site** — nobody can tell what `true` or `null` means without opening the class.
- **Every optional value must be passed every time**, hence the `null, true, null` noise. Overloads for combinations explode: 6 optional fields = 64 combinations.

Your Part A slip: you said "no check for validity or required fields". Not true of this version — the compiler *forces* all ten arguments, and a constructor *can* validate. The real weakness is the reverse: it forces you to pass everything.

Attempt 2 — empty constructor + setters

```
RideRequest r = new RideRequest();
r.setRiderId("U1");
r.setPickup("Airport");           // forgot setDrop()
service.book(r);                  // an invalid request is now in the system
```

1. **The object is usable half-built.** Between `new` and the last setter it exists with no drop location.
2. **No place to validate a rule that spans fields.** `setSeats(3)` is valid alone; `setShareRide(true)` is valid alone; together they break "shared rides allow at most 2 seats". `setSeats` can't check `shareRide` (not set yet), and the caller may set them in either order. **There is no single moment when the object is "finished"** — so there's nowhere to check.
3. **Mutable forever.** It can be changed after validation, after confirmation. Another thread can read it **half-updated** (new seats, old `shareRide`) — a state that was never valid.

Your Part A gap: you got (1) and (3) but missed (2), the cross-field validation point — which is the specific hole the Builder closes.

The solution and structure

Separate collecting the parameters from creating the object. A mutable helper gathers fields step by step; one call — `build()` — validates everything together and produces a finished **immutable** object.

```
public final class Booking {
    private final String guestId;           // 1. ALL fields final
    private final boolean breakfast;
    private final List<String> specialRequests;

    private Booking(Builder b) {           // 2. private ctor – the Builder is the ONLY door in
        this.guestId = b.guestId;
        this.breakfast = b.breakfast;
        this.specialRequests = List.copyOf(b.specialRequests);    // 6. defensive copy
    }

    public String getGuestId() { return guestId; }           // 3. getters only, NO setters

    public static class Builder {           // 4. STATIC nested class
        private final String guestId;           // required: final, in the ctor
        private final LocalDate checkIn, checkOut;
        private boolean breakfast = false;           // optional: NOT final, with defaults
        private final List<String> specialRequests = new ArrayList<>();

        public Builder(String guestId, LocalDate checkIn, LocalDate checkOut) { ... }

        public Builder breakfast(boolean value) {           // 5. each setter returns THIS
            this.breakfast = value;
            return this;
        }
        public Builder specialRequest(String r) { specialRequests.add(r); return this; }

        public Booking build() {           // 6. validate the WHOLE object
            if (!checkOut.isAfter(checkIn)) {
                throw new IllegalStateException("Check-out must be after check-in");
            }
            return new Booking(this);           // ← then CREATE it
        }
    }
}

Booking b = new Booking.Builder("G1", checkIn, checkOut).breakfast(true).build();
```

Detail	Why
static on the nested Builder	A non-static inner class needs an existing Booking, but none exists yet. Same reason as Bill Pugh's Holder. Required.
Private Booking constructor	Guarantees every Booking went through build()'s validation. Works from inside Builder because nested classes can see the outer class's privates.
return this in every setter	What makes chaining work. A void setter ends the chain. The most common mistake when writing a Builder.
Required fields in the Builder's constructor	The compiler then enforces them — they can't be forgotten.
Optional fields not final, with defaults	The Builder is the mutable scratchpad; setters assign later. Only the <i>product</i> is immutable.

Your biggest exercise-20 mistake: you made the Builder's OPTIONAL fields final.

```
private final boolean shareRide;           // ✗ setter can't assign it; ctor never sets it; no default
private boolean shareRide = false;         // ✓
```

With no defaults, seats would be 0 and fail your own validation on every ride that didn't set it. stops was also never created → NullPointerException on stops.add(...).

Also: build() validated and then **returned nothing** — a compile error, and no object would ever be created. The last line must be return new RideRequest(this); (the compute-and-discard habit in a new place). And private static class Builder makes the Builder unusable from outside, so no one could ever create the object.

Where does each rule go?

The test: does the rule involve **one field** or **more than one**?
One field → check it in its own setter and **fail fast**, at the moment of the mistake.
Two or more fields → check in build(). A setter can't know what the other field will eventually be, and the caller may set them in either order.
Depends on "now" → build(), so it's checked when the object is finalised.

Rule	Exercise 20 rule	Where, and why
R1	seats must be 1–4	setter — one field
R2	shared ride allows at most 2 seats	build() — two fields (you missed this one entirely)
R3	pickup ≠ drop	build() — two fields
R4	scheduledAt must be in the future	build() — depends on <i>now</i> ; also null-check, it's optional
R5	at most 3 stops	addStop() — one field
R6	petFriendly can't combine with shareRide	build() — two fields

```
public Booking build() {
    if (shareRide && seats > 2)    throw new IllegalStateException("Shared rides allow at most 2 seats");
    if (pickup.equals(drop))      throw new IllegalStateException("Pickup and drop must differ");
    if (scheduledAt != null && !scheduledAt.isAfter(LocalDate.now()))
        throw new IllegalStateException("Scheduled time must be in the future");
    if (shareRide && petFriendly)  throw new IllegalStateException("Pet-friendly rides cannot be shared");
    return new RideRequest(this);
}
```

Recurring: pickup == drop compares references, not text → pickup.equals(drop) . And currentTime > scheduledAt doesn't compile on Strings — use LocalDateTime with isAfter/isBefore .

Validation REJECTS bad input; it never quietly repairs it. (From the jumbled exercise, where you set meetingHours = 0 for a non-cabin booking instead of throwing. The member thinks they booked meeting hours and silently got none.)

Immutability

Your doubt: "what does immutable mean and how does Builder ensure it? Can I set more properties after build()?"

Immutable = cannot be changed after creation. You already use one daily:

```
String s = "hello";
s.toUpperCase();           // does NOT change s
System.out.println(s);    // hello
String t = s.toUpperCase(); // a NEW String; the old one is untouched
```

A class becomes immutable through choices, not a keyword: **private + final fields, set once in the constructor, getters only, no method assigns to a field.** Then no line of code anywhere can change it.

After build() , you cannot set anything. There are two different objects:

	Builder	The built object
Mutable?	yes — has setters	no — getters only
Lifespan	short: only while assembling	long: used across the system
Purpose	a scratchpad for collecting values	the finished result

```
booking.setBreakfast(false); // x COMPILE ERROR – Booking has no setters
// need a different booking? build a NEW one; the original is untouched
```

Analogy: the Builder is an order form you can scribble on and correct. build() prints the receipt. You can keep editing the form; the printed receipt never changes.

The list trap: a final collection field is not immutable

Your doubt: "a final field holding a collection isn't immutable unless copied — I didn't understand this at all."

A variable doesn't *contain* a list; it holds an **arrow pointing at** one:

```
specialRequests —————> [ "quiet room" ]
  (the final field)           (the list object)

requests = new ArrayList<>(); // x blocked by final – can't move the arrow
requests.add("late checkout"); // ✓ ALLOWED – final does NOT lock the list's CONTENTS
```

The bug, if the constructor copies only the arrow (this.stops = b.stops;): the builder and the built object point at **the same list**.

```
builder.addStop("Mall");
RideRequest r = builder.build();
builder.addStop("Station"); // edits the builder AFTER build()
r.getStops();               // ["Mall", "Station"] ← the "immutable" object changed!
```

The fix: this.stops = List.copyOf(b.stops); — an independent **and unmodifiable** snapshot. Now r.getStops() is [Mall] forever, and even r.getStops().add(...) throws.

Rule: an immutable object holding a collection must keep **its own locked copy**, never a collection someone else can still reach. ✓ **You traced both cases of this correctly in Part C.**

Thread safety

Your doubt: "how is it thread safe — two threads make different objects, but they can make the same object with all the same info?"
Two separate objects with identical data don't interfere at all. Thread-safety problems need **several threads using the SAME object while at least one CHANGES it.**

Hazard	Needs	Possible on an immutable object?
Lost update (counter++)	a write	✗ no writes exist
Check-then-act	a write between check and act	✗
Seeing a half-finished change	a change in progress	✗ nothing ever changes

Every hazard needs *someone changing something*. An immutable object never changes, so **any number of threads can read it with no locks.** (Java also

guarantees `final` fields are correctly visible to all threads once the constructor finishes — another reason for `final`.)

Your Q7 answer was reversed. You said the *builder* can be shared. It's the opposite:
The built object CAN be shared — it never changes.
The builder must NOT be shared — it's mutable; two threads calling `.addStop()` on one builder corrupt each other. Your reasoning ("it's just a local copy") is why it doesn't *need* to be thread-safe, not why it can be shared.
Also wrong reason: the product isn't thread-safe "because it's created via the builder" — it's thread-safe **because it's immutable**. The builder is just how you get there.

Say it: "The builder is mutable and not thread-safe, but it's used locally by one thread and discarded; the object it produces is immutable, so that's what gets shared."

When NOT to use a Builder

Situation	Use instead
Few fields, all required	a plain constructor: <code>new CartItem(name, price)</code>
Pure data carrier, all required	a record: <code>record Money(String currency, long amount) {}</code>
Choosing WHICH type to create	a Factory
2-3 optional fields	a couple of constructors, or sensible defaults

Rule of thumb: reach for a Builder at **4+ parameters, especially when several are optional**, or when you want immutability with cross-field validation. ✓
You answered Q8 correctly for both `CartItem` and `Money`.

Note: you don't need a Builder to get immutability. `final` fields + no setters, or a `record`, give the same thing more simply for small all-required objects.

	Factory	Builder
Answers	Which class do I create?	How do I assemble this one object?
Typical call	<code>factory.forMode("EXPRESS")</code>	<code>new Booking.Builder(...).breakfast(true).build()</code>
Result	one of several types	always one type, configured

✓ **Your Q9 answer was right:** the `String` → fare strategy is the Factory's job; the many-option `RideRequest` with validation is the Builder's. They combine: build the request, then use its type to pick the strategy.
Where you've met it: `StringBuilder` (mutable assembly → one immutable `String`), `Stream.builder()`, `HttpRequest.newBuilder()...build()`, `ProcessBuilder`.
Lombok's `@Builder` generates all of it from one annotation — write it by hand in an interview, but mention Lombok.

Interview framing: "Booking has four required fields and several optional ones, so I'd use a Builder. Required fields go in the builder's constructor so they can't be forgotten, optional ones get named chained setters with defaults, and `build()` validates cross-field rules like check-out after check-in. The result is immutable, which also makes it safe to share across threads."

7. Observer

The problem

When one thing happens, **several other parts must react** — and more reactions keep being added.

```
public void book(Booking booking) {
    // ... booking logic ...
    sms.send(booking.getPhone(), "Booked!");
    email.sendReceipt(booking);
    analytics.recordBooking(booking);
    loyalty.addPoints(booking.getUserId(), 50);
}
```

1. **SRP** — BookingService should know how to book, not who cares about bookings. (Name the actors: comms, analytics, loyalty — three teams whose changes all land in one method.)
2. **OCP** — push notifications, then fraud checks, then WhatsApp: each means **editing** `book()` and adding another constructor parameter.
3. **Tight coupling / DIP** — it depends on four concrete services and can't be tested without them.
4. **No isolation** — if SMS throws, nothing after it runs, and the exception escapes.

Your Part A wording: you said they fail "because all of them are synchronous calls." Synchronous isn't the cause — a synchronous loop *with a try/catch per listener* has no such problem. The cause is **no isolation**. Async is one fix; isolation is the essential one.

Q3 — the trace you should give: line 1 (`setStatus("DELIVERED")`) **already ran**. Then SMS throws, so push/tablet/stock never run and the caller sees a **failure**. The system is now **inconsistent**: the order IS delivered, but the caller was told it failed, the tablet never updated, and stock stays reserved forever. If the caller retries, it re-delivers an already-delivered order. **Side effects must never be able to fail the main action**.

The core insight: the thing that changed shouldn't have to know who's interested. Interested parties **register themselves** — like a newsletter: the publisher doesn't know its readers, people subscribe and unsubscribe, and each issue goes to whoever is on the list.

The structure

```
interface BookingListener {                                // 1. Observer
    void onBookingConfirmed(BookingConfirmedEvent event);
}

class BookingService {                                    // 2. Subject
    private final List<BookingListener> listeners = new CopyOnWriteArrayList<>();

    public void subscribe(BookingListener l) { listeners.add(l); }
    public void unsubscribe(BookingListener l) { listeners.remove(l); } // remove, NOT add!

    public void book(Booking booking) {
        // ... booking logic ...
        notifyListeners(new BookingConfirmedEvent(...));
    }

    private void notifyListeners(BookingConfirmedEvent event) { // private: outside code
        for (BookingListener listener : listeners) {           // must not fake an event
            try {
                listener.onBookingConfirmed(event);
            } catch (Exception e) {
                log.error(listener.getClass().getSimpleName() + " failed", e); // isolate, keep going
            }
        }
    }
}

class SmsNotifier implements BookingListener {             // 3. ConcreteObserver
    private final SmsSender sms;
    public SmsNotifier(SmsSender sms) { this.sms = sms; }
    @Override public void onBookingConfirmed(BookingConfirmedEvent e) { sms.send(e.phone(), "..."); }
}

// wiring, once at startup:
bookingService.subscribe(new SmsNotifier(smsSender));
bookingService.subscribe(new LoyaltyPointsAwarder(loyaltyService));
```

Subject is one, observers are many, and the subject depends only on the observer interface. Adding WhatsApp = one new class + one subscribe line; `BookingService` never changes.

The details interviewers probe

1. Push vs pull — and neither is polling

Your doubt: "push means the subject triggers a function; pull means after a constant time it fetches data?" Push is right. What you described as pull is **polling** — and polling is what Observer *replaces*.

	Who starts it	When	What's sent
Push	subject	immediately, on change	the data: <code>listener.onConfirmed(event)</code>
Pull (Observer)	subject	immediately, on change	only "something changed"; the observer then fetches what it needs
Polling (NOT Observer)	the other side	on a timer, change or not	nothing — it keeps asking

Polling is wasteful (asks when nothing changed) and slow (a change waits for the next tick). **Push vs pull is about what the notification carries; both notify immediately.** Push an immutable event object — the usual middle ground.

2. The event object

```
record BookingConfirmedEvent(String bookingId, String userId, String phone,
                             double amount, Instant confirmedAt) { }
```

- **It must say WHICH entity changed.** Without an id, a listener can't act (which order? which member?).
- **Adding a field doesn't break every listener's signature** — listeners that don't care ignore it.
- **It must be immutable.** The same object goes to every listener; a mutable one lets listener A change what listener B sees.

Your exercise-21 miss: `void onStatusUpdated(String status)` — the listeners only received "CANCELLED". **Which order?** Stock can't release the right items; the tablet can't show which order; the notifier can't text the right person. Also: use an **enum**, not a String — "CANCELED" (one L) compiles and matches nothing.

3. Isolation — one failure must not affect the rest

Without a per-listener try/catch, the loop stops at the first exception **and** it escapes into the main method, so an action that succeeded looks like it failed. **Observers are side effects; their failure shouldn't undo the main action.**

4. Synchronous vs asynchronous

Your doubt: "explain the executor code, and what happens if a listener throws?"

```
private final ExecutorService executor = Executors.newFixedThreadPool(4); // ONE pool, 4 workers

private void notifyListeners(BookingConfirmedEvent event) {
    for (BookingListener listener : listeners) {
        executor.submit(() -> { // packages a task and QUEUES it; returns at once
            try {
                listener.onBookingConfirmed(event);
            } catch (Exception e) {
                log.error("Listener failed", e); // MUST catch INSIDE the task
            }
        });
    }
}
```

Think of a kitchen with 4 cooks and an order rail: you don't hire a cook per order; tasks go on the rail and whoever is free takes the next. The loop finishes instantly having only *queued* work.

The trap: `submit()` returns a `Future` and **stores any exception inside it**. If nobody calls `future.get()`, the exception is **silently swallowed** — no log, no stack trace, notifications quietly stop while the system looks healthy. (`execute()` instead prints it and kills the worker.) **With async code, catch and log inside the task — nobody outside is listening.**

	Synchronous	Asynchronous
Main action speed	waits for every listener	returns immediately
Order of listeners	predictable	not guaranteed
Failures	visible to the caller	invisible unless logged in the task
Durability	—	"success" no longer means "done": a crash loses the in-memory queue
Thread safety	simpler	listeners run concurrently → event must be immutable; stateful listeners must be protected

Better than async-everything: make only the **slow** listener asynchronous. Keep the fast ones synchronous, where order and errors stay simple. ✓ **Your Q9 answer got the executor and the thread-safety requirement and the ordering tradeoff.** Also shut the pool down on exit (`executor.shutdown()`), or its threads keep the app alive.

5. Thread-safe subscribing

The listener list is shared mutable state. Subscribing while another thread iterates it throws `ConcurrentModificationException` on a plain `ArrayList`. `CopyOnWriteArrayList` copies its internal array on every add/remove, so an in-progress loop keeps reading its **snapshot** and never breaks. The new listener misses the event in flight and gets the next one. Writes are expensive, reads are cheap and lock-free — perfect when you subscribe rarely (startup) and notify constantly. ✓ **Your Q6 answer was right.**

6. The lapsed listener (a memory leak)

The subject holds a reference to every observer, so an observer that never unsubscribes **can never be garbage-collected**. Common with UI screens and per-session objects. **Anything that subscribes temporarily must unsubscribe.**

7. Keep listener interfaces focused (ISP)

One fat interface with `onConfirmed` / `onCancelled` / `onRefunded` forces every observer to implement all three. Either one small interface per event, or `default` empty methods (the cache-hooks approach — fine here, since these are optional notifications).

When NOT to use Observer

The test: "If this step fails, is the main action still OK?"

Yes (email, coffee, analytics) → side effect → make it a listener.

No (access card, stock release) → it's part of the action → **call it directly**, and let its failure stop the operation.

- **Required steps in a sequence** ("reserve seat → charge card → confirm") are a flow, not a broadcast. Observer gives no ordering guarantee and hides the flow.
- **Debuggability cost:** reading `book()` no longer tells you everything that happens. Worth it only when the reactions really are independent.

Exercise 21 Q10 — critical stock release. Two defensible answers:

A. Take it out of Observer. Release stock *inside* the cancel transaction, so either both happen or neither. Tradeoff: coupling — acceptable, because it's a required step.

B. Keep Observer but make it reliable. Two holes in a plain retry: **(1)** in-memory retries are lost on a crash → write pending work to a durable outbox/queue with a worker and a failure alert; **(2)** a retry can run **twice** (the call succeeded but the reply was lost) → the operation must be **idempotent**. **Retries are only safe on idempotent operations.** ✓ **You reached "Observer isn't right here" and the retry idea on your own.**

Q11 — why a mutable event is wrong

- ✓ **Your answer was strong:** threads can corrupt it, **and** listeners must stay independent. Two additions:
- **It breaks even single-threaded.** Observer promises no order, so if the receipt listener was subscribed before the loyalty listener, the receipt prints the **undiscounted** amount. Correctness would depend on the order of `subscribe` lines at startup.
 - **Where the discount belongs:** compute it **before publishing**, in the subject. The event then carries the final amount and every listener sees the same value. **An event is a record of something that already happened.**

Observer in Spring

Your doubt: "explain how this pattern is used in Spring." Spring keeps the listener list for you.

Role	Hand-written	Spring
Event	a record	the same record, unchanged
Notify	your loop	<code>ApplicationEventPublisher.publishEvent(event)</code>
Observer	<code>implements BookingListener</code>	any bean method with <code>@EventListener</code>
<code>subscribe()</code>	an explicit call	automatic — Spring finds them at startup and matches by parameter type
The list	<code>CopyOnWriteArrayList</code>	kept inside the application context

```
@Service
public class BookingService {
    private final ApplicationEventPublisher publisher; // injected
    public void book(Booking b) {
        repository.save(b);
        publisher.publishEvent(new BookingConfirmedEvent(...)); // no SMS/email/loyalty deps here
    }
}

@Component
public class SmsNotifier {
    @EventListener // matched by the parameter TYPE
    public void onBookingConfirmed(BookingConfirmedEvent event) { ... }
}
```

The catch: Spring events are SYNCHRONOUS by default. `publishEvent()` calls each listener on the same thread and waits — so a slow listener slows `book()`, and **an exception in a listener travels back into `book()`**. You must opt out.

```
@EnableAsync // once, on the application class
@Async @EventListener // this listener now runs on Spring's thread pool
public void onBookingConfirmed(BookingConfirmedEvent e) { ... } // still log failures INSIDE

@EventListener(condition = "#event.newStatus.name() == 'CANCELLED'") // filter outside the body
@Order(1) @EventListener // ordering (meaningless for @Async)

@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT) // ← the production one
public void onBookingConfirmed(BookingConfirmedEvent e) { sms.send(...); }
```

Why `@TransactionalEventListener` matters: if `book()` saves inside a transaction, publishes, and then the transaction **rolls back**, a plain listener has already sent "Booking confirmed" for a booking that doesn't exist. `AFTER_COMMIT` runs only once the write is permanent. Mentioning this shows production experience.

Spring also publishes its own events (`@EventListener(ApplicationReadyEvent.class)`). Note `java.util.Observer` is **deprecated** since Java 9 — write your own interface.

	Observer	Pub/Sub
Who knows whom	the subject holds the observer list directly	publishers and subscribers never know each other
Middleman	none	a broker (Kafka, RabbitMQ)
Scope	inside one application (one JVM)	across services and machines
Delivery	usually immediate, in-process	async, can be persisted and retried

Interview framing: "Confirmation triggers several independent side effects and more will be added, so I'd use Observer. The service publishes an immutable event to listeners it knows only by interface — a new reaction is one class plus a subscribe call. I'd isolate each listener with try/catch, use CopyOnWriteArrayList for safe subscription, and make slow listeners async. Anything the booking *depends* on, like access-card activation, I'd call directly instead."

Exercise 21 — your other mistakes

- ✗ The four listener classes had **no implements StatusListener**, and their methods weren't `public` — so `subscribe(new RiderApp(...))` wouldn't compile.
- ✗ `OrderService` held `private String status` — **one service per order**, so all four listeners would need re-subscribing for every order. The status belongs to the **order** (a `Map<String, OrderStatus>`), not the service. Same per-request-state lesson as mixed exercise 2.
- ✗ `unsubscribe()` called `add()` — a listener would then fire twice.
- ✗ `catch { // throw exception }` — needs a type, and rethrowing defeats requirements (a) and (b). **Log and move on.**
- ✗ No transition guard: `CANCELLED` after `DELIVERED` was allowed. (Managing legal transitions is exactly what the **State** pattern does — next session.)
- ✓ Per-listener try/catch, `CopyOnWriteArrayList`, filtering inside each listener, correct wiring.

One more nuance: `updateStatus` is itself a check-then-act (read old status, check, write new). Two threads on the same order could both pass the check — use `map.compute(...)` or a per-order lock. Worth saying out loud even if you don't code it.

8. Jumbled Exercise 1 — Co-working Space Booking

First exercise with **no hints** about which pattern applies anywhere — requirements written the way a product manager writes them.

Part A — the decisions

Req	Answer	Why
R1	Builder	4 required + 6 optional fields, cross-field rules, and immutable once confirmed ✓
R2	Strategy + Factory	a formula per type; type arrives as a String; new type twice a year ✓
R3	One shared instance + ConcurrentHashMap	"the three systems disagreed in the audit" = there must be one source of truth ✓
R4	Observer for email/coffee/analytics · direct call for the access card	see the trap below
R5	No pattern — a constant	one national GST rate, set by law ✓
R6	No pattern — a plain method	one rule, unchanged in five years ✓ (this was a deliberate trap — it looks like Strategy)
R7	No pattern — a plain class, per booking, separate from Booking	the booking is immutable but the log changes all day → it can't be a field of Booking
R8	Nothing	no ticket, no date — speculative ✓

A9 — the main trap: the access card. Every other line of R4 is a side effect; if the email fails, the member still has their desk. But "*without it, the member physically cannot enter*" — a confirmed booking that won't open the door isn't confirmed. It **looks like** one more listener. It **must be** a required step, called **directly and first**. As a listener, its failure would be caught and logged while `confirm()` still reported success — the member is told "you're booked", then gets stuck at the door. Listeners also have no guaranteed order and may be async. This is the same judgement as exercise 21 Q10 (critical stock release).

A10 — shared vs per booking ✓ you got this right

Object	Scope	Why
Occupancy tracker	shared	must be one source of truth; thread-safe inside
Pricing strategies + factory	shared	stateless
BookingService	shared	stateless; the booking arrives as a parameter
Listeners	shared	stateless; the event carries the data
Booking	per booking	one member's booking; immutable
Booking.Builder	per booking	mutable, used once by one thread, discarded
VisitLog	per booking	changes all day

Your doubts, answered

"Is a cabin booking different from a workspace booking?" No — one Booking class; the workspace type is a **field**. "Cabin booking" = a booking whose type is CABIN. That's why "meeting hours only for CABIN" is a two-field rule.

"Where is the visit log used?" Not in `confirm()` — that happens when they *book*, maybe days ahead. The log is written **on the day**, by a `GateService.onEntry/onExit`, which also updates the shared occupancy tracker. It needs no concurrent collection (one member, one booking); the tracker does (every gate in the branch).

"Where does the late-cancellation fee go?" In the `cancel()` path, never in `confirm()`. Different operations live in different methods. It needs the confirmed amount, so store that when confirming.

The corrected confirm() — the shape is the answer

```
public double confirm(Booking booking) {
    // 0. the rule the Booking couldn't check itself (needs branch data)
    if (booking.hasParking() && !branches.hasParking(booking.getBranchId())) {
        throw new IllegalStateException("Branch has no parking");
    }
    // 1. price, including GST
    double base  = pricing.forType(booking.getWorkspaceType()).priceFor(booking);
    double amount = base + base * GST_RATE;

    // 2. REQUIRED step – called DIRECTLY and FIRST. If it throws, nothing is announced.
    accessCards.activate(booking.getMemberId(), booking.getBranchId(), booking.getDate());

    // 3. record it as confirmed (cancel() will need the amount)
    confirmedAmounts.put(booking.getBookingId(), amount);

    // 4. ONLY NOW announce to the optional side effects
    notifyListeners(new BookingConfirmedEvent(booking.getBookingId(), ..., amount));
    return amount;
}
```

Validate → price → required step → record → announce. Each later step only happens if every earlier one succeeded.

Your Part B mistakes

- ✗ **THE BIG ONE:** you called `notifyListeners(...)` **before** `activateMemberShip(...)` — the exact reverse of your own Part A decision. If activation fails, the member has already been emailed "you're booked" and given a coffee for a booking that can't get them inside.
- ✗ The access card was an **empty method inside BookingService**. It's an external system → an **interface injected through the constructor** (Phase 2 rule), and it needs `member/branch/date`, not the event.
- ✗ The notify loop **lost its try/catch** (you had it right in exercise 21), and there was **no `subscribe()`** at all, so no listener could ever be added. `ArrayList` → `CopyOnWriteArrayList`.
- ✗ `calculatePrice(WorkspaceType type)` — the strategy **already is** the type, so passing it tells it nothing. A formula needs the booking's details → `priceFor(Booking booking)`. Also: it was called with no argument, the factory took a `String` but got the enum, and **GST was declared and never used** (`private final static gst = 18`: no type, and as 18 it would add 1800%).
- ✗ `booking.workspaceType` — private fields read from outside. Use the getters. *Your most persistent habit since Phase 1.*
- ✗ Builder declared **no fields at all**; `workspaceType` never assigned in the constructor (a final field must be); `private static class Builder; this.data = data; dates as Strings` compared with `<`.
- ✗ `if (type != CABIN) { meetingHours = 0; }` — **silently repaired invalid input** instead of throwing.
- ✗ Tracker: `public getInstance()` (no static, no return type), `Holder` field with no type, constructor missing `()`, `computeIfAbsent` on a **read**, `Math.Max`, undefined `getBookingsCount`.
- ✗ `HotDeskPricingStrategy` never closed its brace, so every class after it was **nested inside it**.
- ✗ `LateCancellationPolicy {}` — empty; Q5 asked for the rule.
- ✓ The enum, the immutable event record, and all three listeners (implements + public method + service injected) were correct — all previously-missed things now fixed.

Part C — the pressure answers

Q6 — add PHONE_BOOTH (priced per hour) ✓ you spotted the new data

File	Change
WorkspaceType	add the constant
PricingFactory	one registration line
Booking + Builder	add optional <code>bookedHours</code> + a cross-field rule in <code>build()</code>
<i>new:</i> PhoneBoothPricing	<code>rate × bookedHours</code>

Booked hours, not hours spent — pricing happens at confirm, before they arrive. And it's **not a zero-edit change**, which is the honest answer: **OCP protects you from new behaviour; it does not protect you from new data.**

Q7 — referral bonus ✓ One new listener class + one subscribe line. `BookingService` and every existing listener: **untouched**. Note the event needn't change either — the listener gets a `ReferralService` in its constructor and looks up the referrer from `event.memberId()`. **Listeners fetch extra data themselves**, so the event stays small.

Q8 — email provider down for an hour ✓ Bookings still confirm, the card is already active, coffee and analytics still run, the failure is caught and logged. Two additions: **(1)** those emails are **lost** — logging isn't retrying; add a retry queue (durable outbox). **(2)** a "down" provider often **hangs** until timeout, so every confirm waits — argue for making the email listener `async` with a short timeout.

Q9 — access-card system down ✓ `confirm()` throws at the activate call. Nothing is announced, nothing is recorded, and the member finds out **immediately** instead of at the door. A clean failure beats a fake success. Two extras: if a *later* required step fails, **compensate** (deactivate the card); and activation must be **idempotent** so a retry is safe. *Business alternative:* accept as "pending activation" and retry in the background — better availability, but entry isn't guaranteed. Naming it shows you see it as a business decision.

Q10 — one server becomes three ✓ **you got the key insight A Singleton is one instance per JVM.** Three servers = three trackers, each counting only the gates that reached it — the audit problem returns. **The fix:** move the state out of the servers into a shared store. **Redis** has atomic `INCR/DECR/GET` — it does across servers what `AtomicInteger` did inside one JVM. Behind the same `OccupancyTracker` interface, this is one new class + wiring (the DIP payoff). Also breaks: the `confirmedAmounts` map, visit logs, idempotency sets — anything in one server's memory that must be shared. **Doesn't break:** strategies, factory, listeners, `BookingService`. **Stateless objects scale for free; shared state must move to a shared store.**

9. Revision Tools

Updated mistake summary (Builder · Observer · jumbled 1)

#	Habit	Status	Fix
1	Decided correctly in Part A, then contradicted it in code	worst	After coding, walk your Part A list and point to the exact line that implements each decision. If you can't point to it, it isn't done.
2	Missing return types / <code>static</code> on <code>getInstance()</code>	8x	Modifiers → return type → name. Scan every method before submitting.
3	Reading private fields from outside instead of the getter	recurring	<code>booking.getMemberId()</code> , never <code>booking.memberId</code> .
4	Classes not <code>implements</code> -ing their interface; impl methods not <code>public</code>	3x	Interface methods are public; an impl can't reduce visibility.
5	Singleton/Builder boilerplate slips (<code>private</code> Builder, missing <code>static</code> on Holder, no private ctor)	recurring	Builder: <code>public static class</code> . Holder: <code>private static class</code> + typed <code>static final</code> field.
6	Skipping a requirement (R2 in ex 20; event fields in ex 21; R6 in jumbled)	4x	Tick every numbered requirement against a line of code.
7	Regressing a lesson already learned (try/catch isolation)	new	Re-read the previous exercise's notes before a related one.
8	Shared service holding per-request state	3x	Per-request data arrives as a parameter , never a field.
9	<code>==</code> on Strings; Strings compared with <code><</code>	recurring	<code>.equals()</code> ; <code>LocalDate/LocalDateTime</code> with <code>isBefore/isAfter</code> .
10	Giving an abstraction data it doesn't need (strategy taking its own type)	new	Pass what the job needs, not what identifies the class.
—	Spoken reasoning on judgement questions	strong	Keep answering Part C by voice — it's the closest thing to a real round.

Requirement → pattern cheat sheet (all five patterns)

If the requirement says...	Reach for
"supports X, Y and Z ways of doing...", rules vary, algorithm swappable	Strategy
type arrives as a String/config at runtime; "business logic must not name..."	Factory (switch if small & fixed; registry if it grows)
objects must come as a matching set	Abstract Factory
"every part must see the same...", "the screens disagreed", "exactly one pool/registry"	Singleton / one injected instance + thread-safe internals
a few required fields + many optional ; rules spanning fields; "cannot change once confirmed"	Builder
"when X happens, also do A, B, C" and more will be added	Observer
...but one of those reactions is required for the action to be meaningful	Direct call, first, not a listener
per-request / per-booking data that accumulates	No pattern — plain class, fresh per use
one variant, confirmed stable (one tax rate, one fee rule)	No pattern — constant or plain method
"maybe one day...", no ticket, no date	Nothing — mention the seam verbally
external system (SMS, payment, DB, access cards)	Interface even with one implementation (testability)
data that changes all day on an otherwise immutable entity	A separate class, linked by id

Interview quick-recall — Builder & Observer

Cover the right column and answer out loud in one or two sentences.

Question	Answer in one breath
What problem does Builder solve?	Objects with a few required and many optional fields, where a big constructor is unreadable and setters leave the object half-built and mutable.
What's wrong with a 10-parameter constructor?	Same-typed arguments can be swapped and still compile, the call site is unreadable, and every optional value must be passed every time.
What's wrong with setters?	The object is usable half-built, there's no moment to validate cross-field rules, and it stays mutable forever.
Where do you validate, setter or build()?	One field → the setter, to fail fast. Two or more fields, or anything depending on "now" → build().
Why is the Builder a static nested class?	A non-static inner class needs an instance of the outer class, and none exists yet.
Why does each setter return this?	So calls can be chained; a void setter ends the chain.
Should the Builder's fields be final?	Only the required ones. Optional fields are assigned by setters later and need defaults.
Can you change an object after build()?	No — it has no setters. Build a new one instead.
What makes a class immutable?	Private final fields, set once in the constructor, getters only, no method writes a field, and defensive copies of any collections.
Why copy a list into an immutable object?	final locks the reference, not the contents; sharing the builder's list lets later edits change the "immutable" object.
Is the Builder thread-safe?	No, and it doesn't need to be — it's local to one thread and discarded. The immutable product is what gets shared.
Why is an immutable object thread-safe?	Every hazard needs a write; nothing ever changes, so any number of threads can read it without locks.
When would you NOT use a Builder?	Few fields, all required — a constructor or record is clearer. Roughly 4+ parameters with several optional is the threshold.
Builder vs Factory?	Factory chooses which type; Builder assembles one type with many options.
A Builder in the JDK?	StringBuilder — mutable assembly, then one call produces the immutable String.
What problem does Observer solve?	One event with several independent reactions that keep being added, without the subject knowing who reacts.
Name the two roles.	Subject holds a list of observers and notifies them; observers implement one interface and react.
Push vs pull?	Push sends the data with the notification; pull says only "something changed" and the observer fetches. Both notify immediately — polling is neither.
What must the event contain?	Which entity changed, plus whatever listeners need — and it must be immutable.
Why immutable events?	The same object goes to every listener, possibly on several threads; a mutable one lets one listener change what the next sees.
What if a listener throws?	With a per-listener try/catch, the others still run and the main action still succeeds. Without it, the loop stops and the caller sees a failure.
Why CopyOnWriteArrayList?	Subscribing while notifying would throw ConcurrentModificationException; it iterates a snapshot. Cheap reads, rare writes — exactly this workload.
What do you gain and lose going async?	Gain speed; lose ordering, visible errors, and delivery guarantees if the process dies.
Why must async tasks catch their own exceptions?	submit() stores the exception in a Future nobody reads, so failures vanish silently.
What's the lapsed listener problem?	The subject holds a reference forever, so an observer that never unsubscribes can't be garbage-collected.
When is Observer the wrong tool?	When the reaction is a required step of the action, or when steps must happen in a guaranteed order.
How do you decide listener vs direct call?	Ask "if this fails, is the main action still OK?" Yes → listener. No → direct call, first.
How do you make a critical reaction reliable?	Durable queue plus retries, and the operation must be idempotent so a retry can't double-apply it.
Observer in Spring?	publishEvent plus @EventListener; Spring keeps the list and matches by parameter type.
Are Spring events async?	No, synchronous by default — a listener exception propagates into the publisher. Use @Async to change that.
What is @TransactionalEventListener for?	Running a listener only AFTER_COMMIT, so you never notify about a booking whose transaction rolled back.
Observer vs pub/sub?	Observer is in-process with a direct list; pub/sub adds a broker so publishers and subscribers never know each other, across services.
Why does a Singleton break on three servers?	It's one instance per JVM, so each server gets its own copy. Shared state moves to Redis or a database.

Pre-submit checklist (use this every time)

1. The self-check that matters most

- ☐ Walk your Part A decisions one by one and **point to the code line** that implements each
- ☐ Tick every numbered requirement against a line of code
- ☐ Re-read your own critique — is every problem you named actually **fixed** in the code?

2. Builder

- ☐ `public static class Builder`, with a `public` constructor
- ☐ Required fields final in the Builder's constructor; optional fields **not final**, with defaults
- ☐ Every setter returns `this`; `build()` ends with `return new X(this);`
- ☐ One-field rules in setters; cross-field and "now" rules in `build()`
- ☐ Validation **throws** — it never silently repairs input
- ☐ Product: all fields final, no setters, private constructor, `List.copyOf` for any collection

3. Observer

- ☐ Event is an immutable record carrying **which entity** changed
- ☐ Every listener class `implements` the interface, with `public` methods
- ☐ `subscribe()` **and** `unsubscribe()` exist — and unsubscribe **removes**
- ☐ `CopyOnWriteArrayList` for the listener list
- ☐ Per-listener try/catch that **logs and continues** (never rethrows)
- ☐ `notifyListeners` is private
- ☐ Required steps called **directly and first**; only then notify

4. Syntax sweep

- ☐ Every method has a return type; `getInstance()` is `static`
- ☐ Getters used from outside, never private fields
- ☐ `.equals()` on Strings; `LocalDate` for dates
- ☐ Every class closes its brace before the next one starts
- ☐ Every constant has a type and is `UPPER_CASE` (a rate is 0.18, not 18)
- ☐ Every injected collaborator is actually **used** in the method body

Status: Strategy ✓ Factory ✓ Singleton ✓ Builder ✓ Observer ✓ • Jumbled exercise 1 ✓

Next: State pattern → **Parking Lot** (first full Phase 5 problem) → Decorator → Vending Machine → Adapter → Elevator → Command → Cache → Chain of Responsibility → ATM → Template Method + Facade → Splitwise → Composite + Proxy → BookMyShow → recognition-level patterns.